

Introduction

The project consists in a partial recreation of the rendering style of the 2018 video game "Return of the Obra Dinn", using C++/OpenGL. RotOD is a first-person mystery puzzle game where you play an insurance investigator in 1807, determining the fate of each crew member aboard an abandoned merchant ship.

Its "1-bit" aesthetic uses black-and-white pixels with dithered shading, creating a high-contrast, hand-drawn look reminding of early computer displays. The style is directly inspired by 1980s Macintosh games (e.g., Dark Castle or The Fool's Errand), which used monochrome graphics and point-and-click interfaces due to hardware limitations.

The game thus uses a classic 1-bit aesthetic with various adjustments, techniques, and methods for a more pleasant experience within a classic first person (fps like) camera.

Out of the various graphical techniques showcased in the game I prioritized what I deem as the most impactful towards the final look and feel of the game. Out of the various rendering and shader techniques, I chose:

- Low Resolution pixelated look — The game's (RotOD) render resolution is locked at 800x450, which is then nearearly upsampled to the screen's resolution.
- Dithering for pixel brightness approximation — In a black/white or 1-bit look, the game uses dithering to emulate the different brightness values of whatever pixel "would" be in the screen.
 - This includes the implementation of multiple dithering techniques, used to create contrast between different types of objects in the scene and as a gameplay feature. For example, people's faces are rendered using a novel dithering technique that better preserves legibility and edge clarity.
 - This also includes the implementation of a novel dithering technique unique to this game's development ("Dither Sphere").
- Stylized object outlines — In RotOD better legibility is achieved through stylized (inspired by 80s games) traced edges within a simple object geometry. This accentuates object's position in the world, making them and their surroundings more visible to the player.
 - As a way to control outline density, it is possible to "disable" or "force" the outline of specific edges in a model.
- Dust and dust clouds — As a clever optimized particle effect, the game utilizes a "dust cloud" effect. The dust clouds are widely used in the game, both as a simple graphical effect and as a gameplay element through dense vision-blocking fogs during certain gameplay moments.

Program outline and Render Pipeline

I recreated these effects through a multi-stage deferred-like pipeline that separates lighting, edge detection, and dithering culminating in a low-resolution, nearest-neighbor upsampled output.

The project uses the core profile of OpenGL 4.1, with the libraries GLFW (3.4.0), GLAD (0.1.36), GLM (1.0.0), Assimp, and stb_image (2.30). Its entire pipeline is based on chained

FBOs, this allows for multi-pass rendering with immediately debuggable and visible intermediate results. This also allows for quick standardized post-processing methods without affecting the main scene.

The pipeline is thus not pure deferred, but a multi-pass forward renderer. This gives simplicity for quickly implementing and experimenting diverse shaders and render passes without managing a complex buffer.

The rendering pipeline is structured this way:

1. Lighting Render Pass — Textures, directional lighting (with shadows), point lights, and dithering get processed. The texture value is inserted in the yellow channel and the lighting value in the green channel, then each channel gets dithered according to the relevant dithering method.
2. Edge Accentuation — Every face of each model gets assigned a pseudo-random unique color, approximately guaranteeing a strong contrast between each face that will facilitate edge detection. Here dust clouds are rendered with a full blue color that will be properly handled in following steps.
 - 2a. Edge Accentuation 2 — Here the wireframe (if enabled), normal textures (used for manual outline creation instead of vertex color), and depth maps are also rendered.
3. Outline creation — The first of the post-processing steps, this uses the result of the previous Render Pass and outputs a white on black outline of the previously accentuated edges. The edge detection is ignored in case
 - 3a. Thin outline creation — If enabled, this uses a different algorithm that includes the wireframe and depth map from the 2a render pass in order to create more accurate 1 pixel thick outline (more like RotOD).
4. Combination pass — Here the results from the outline and lighting render pass get properly combined.
5. Upscale — The result of the combination pass is upscaled using nearest neighbor and gets output to the main buffer.

Code and class structure

The basic structure of the code (template) is taken from the lectures. The code for all of the new classes I have defined is in “include/utils/”:

- camera.h/shader.h — Largely unmodified from the lecture code.
- light.h — Contains classes “DirectionalLight” and “PointLight”.
- mesh.h/model.h — Added a “DrawPoints” function, only used to render dust particles.
- misc.h — Miscellaneous self-contained functions used throughout the code. /misc.cpp contains their code
- rendering.h — Contains the abstract class “RenderPass” and all of the project’s different Render Passes as separate inheriting classes.
- scene.h — Contains the “Scene” class. /scene.cpp is where Scenes are defined (written) and loaded.
- sceneobject.h — Contains the class “Object” for objects contained in scenes.

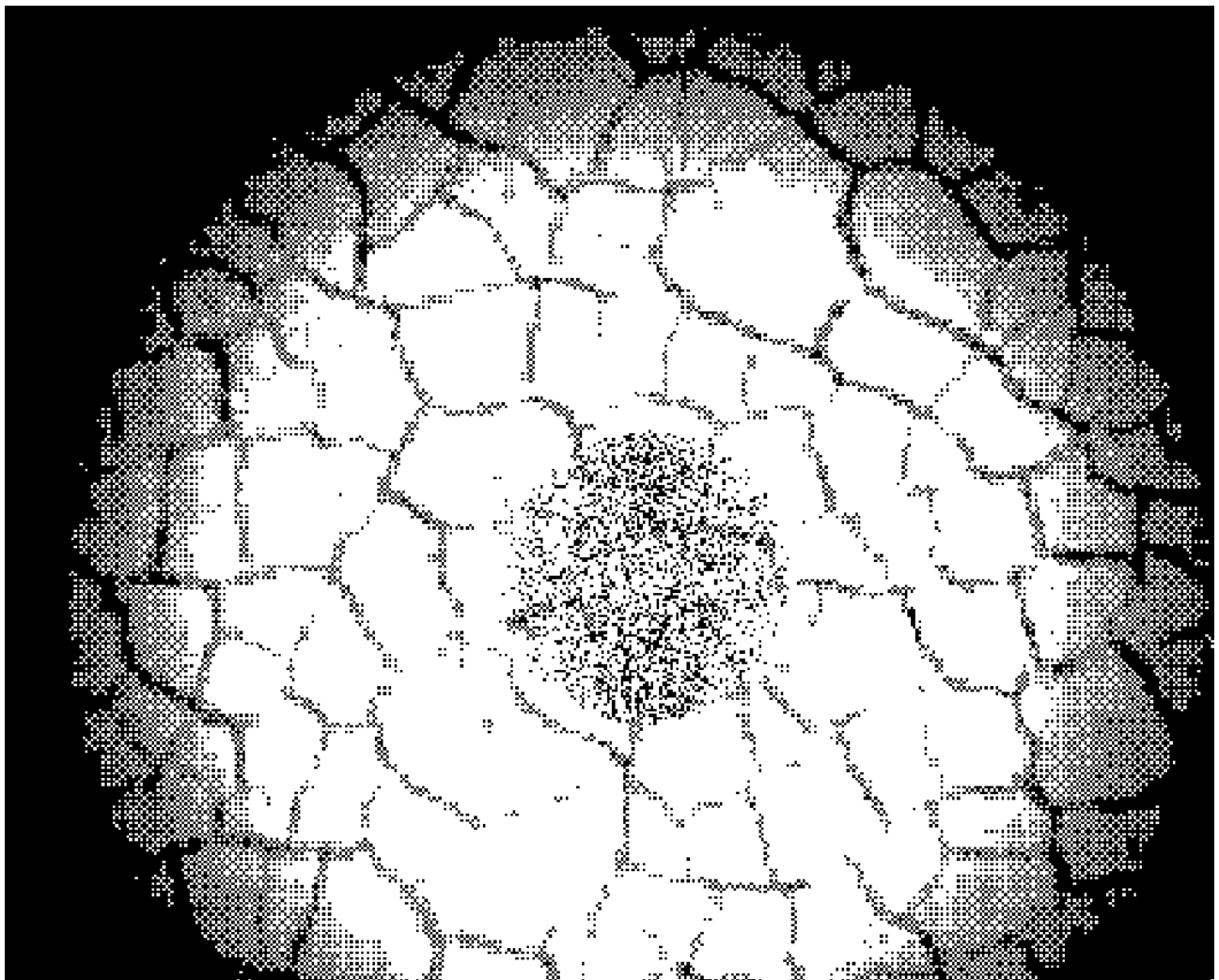
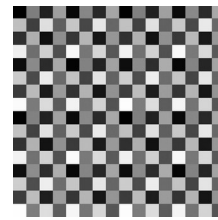
Algorithms and Techniques

Dithering

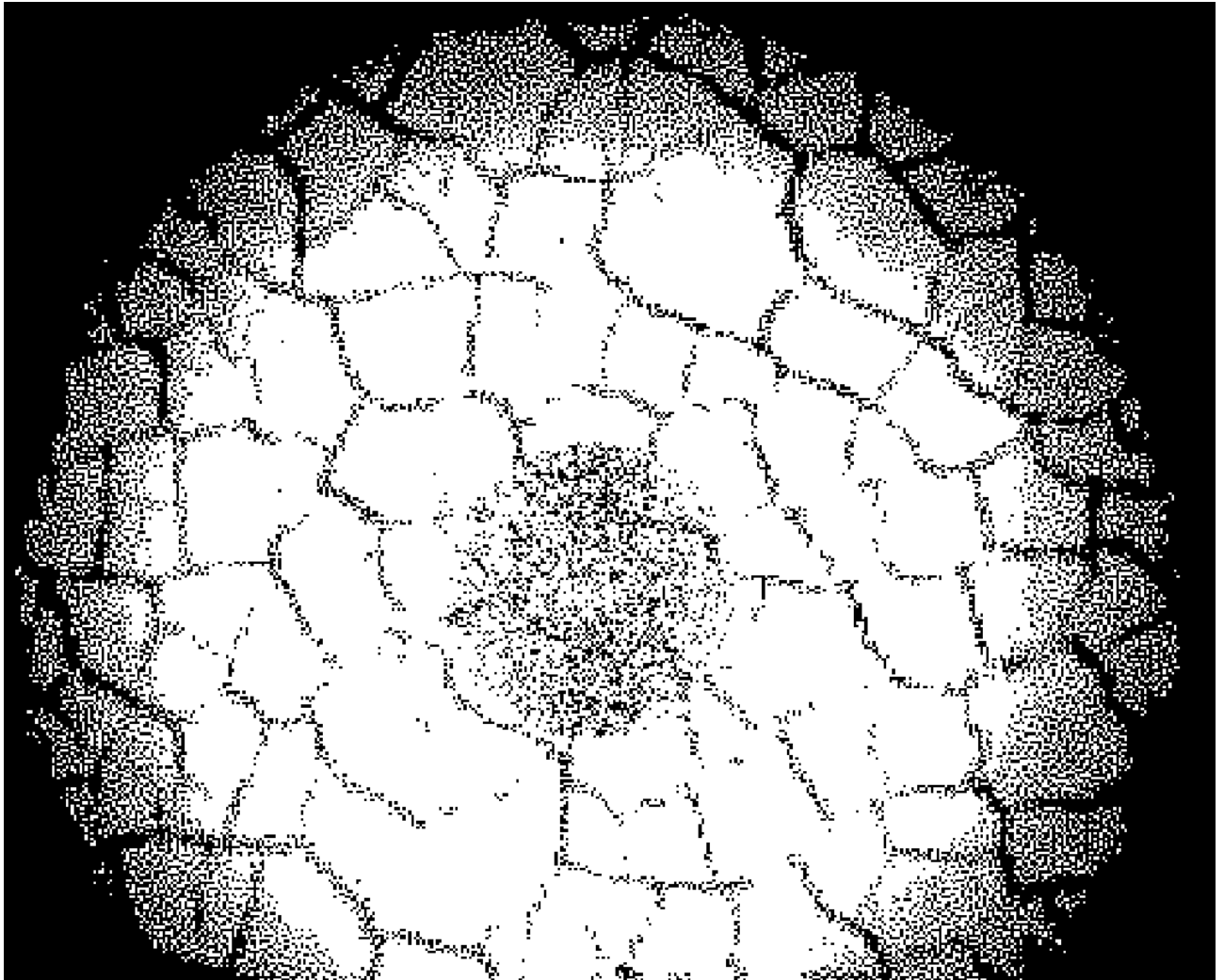
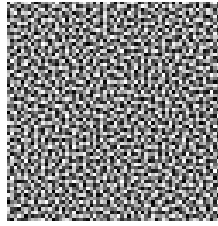
True 1-bit (black/white) rendering loses all shading information, making shapes unreadable. The solution is to use dithering to simulate intermediate grays. Two different dithering types (with a sphere variant for each) have been implemented:

- Ordered Dithering (Bayer) — using a 16x16 bayer matrix we can compare luminance values and create a regular, cross-hatch dither pattern.
- Blue Noise Dithering — this achieves a more “random” and organic film-grain look. In this project I use a precomputed 64x64 blue noise image.

ORDERED DITHERING



BLUE NOISE



Every object in the scene can use a single dithering option, implemented in the code through an enum "NoiseType" field in the "Object" class. The noise/dither textures are loaded and contained in the LightingPass class.

As an optional boolean field in the LightingPass class, you can enable my implementation of the Dither Sphere. I implement this "dither sphere" by rendering an icosphere from the inside with the dither texture mapped via UVs. This sphere is rendered from the camera's position without translation, rotating with the view. The resulting image is a view-aligned dither pattern.

In rendering.h, class LightingPass:

```

void update_blue_sphere_cubemap(Scene* scene){
    glDepthFunc(GL_LESS);
    glDepthMask(GL_TRUE);
    glBindFramebuffer(GL_FRAMEBUFFER, blue_sphere_fb);
    glViewport(0, 0, 1280, 720);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    blue_sphere_shader.Use();

    glActiveTexture(GL_TEXTURE0);
    glBindTexture(GL_TEXTURE_2D, blue_noise); // bind blue noise texture
    blue_sphere_shader.set_uniform1i("noise",0);

    blue_sphere_shader.set_uniform1f("tile",pvbl); // set tiling factor for blue noise

    // set dither sphere position to camera position, so that it always surrounds the camera
    dither_sphere.pos = scene->camera.Position;

    glm::mat4 viewRot = glm::mat4(glm::mat3(scene->camera.GetViewMatrix())); // rotation only
    glm::mat4 model = glm::mat4(1.0f); // identity

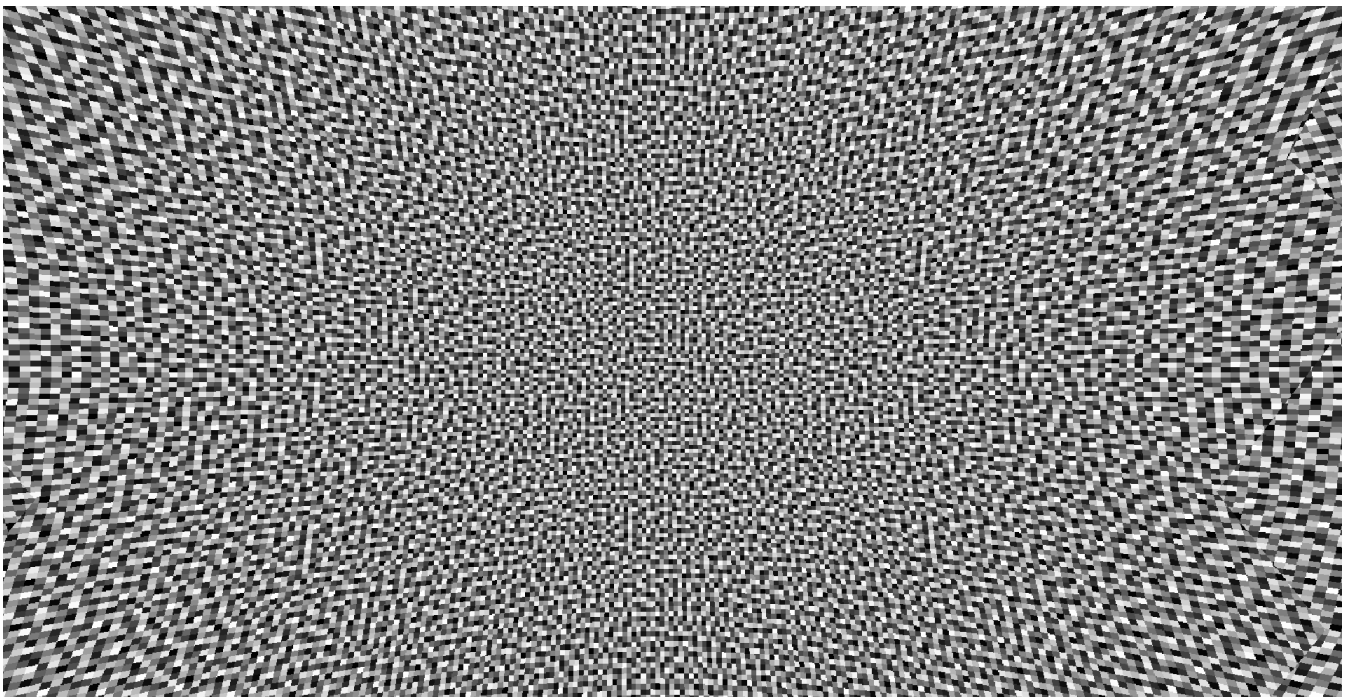
    glm::mat4 mvp = scene->projection_matrix * viewRot * model;

    blue_sphere_shader.set_uniformMatrix4fv("mvp", mvp);
    dither_sphere.model.Draw();
    glBindTexture(GL_TEXTURE_2D, 0);
}

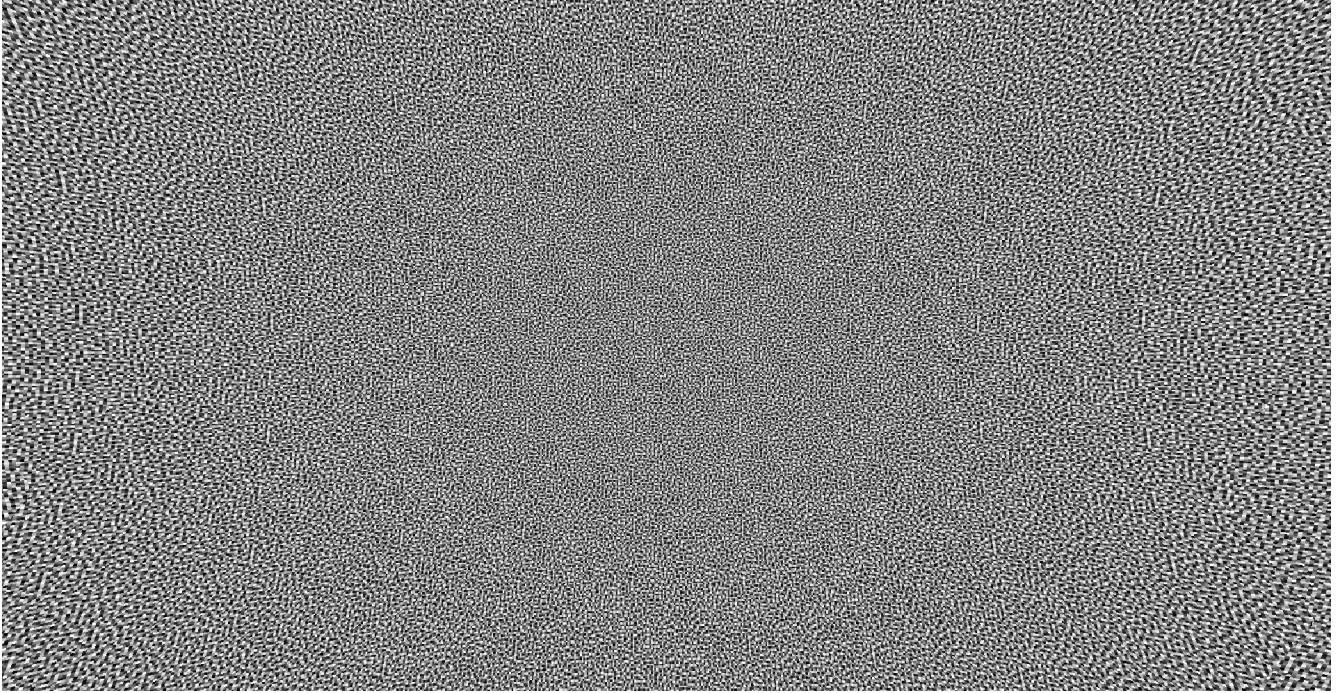
```

Optionally I can change the pattern repetition factors for the dither textures, therefore being able to choose between a more stable but less precise image or viceversa:

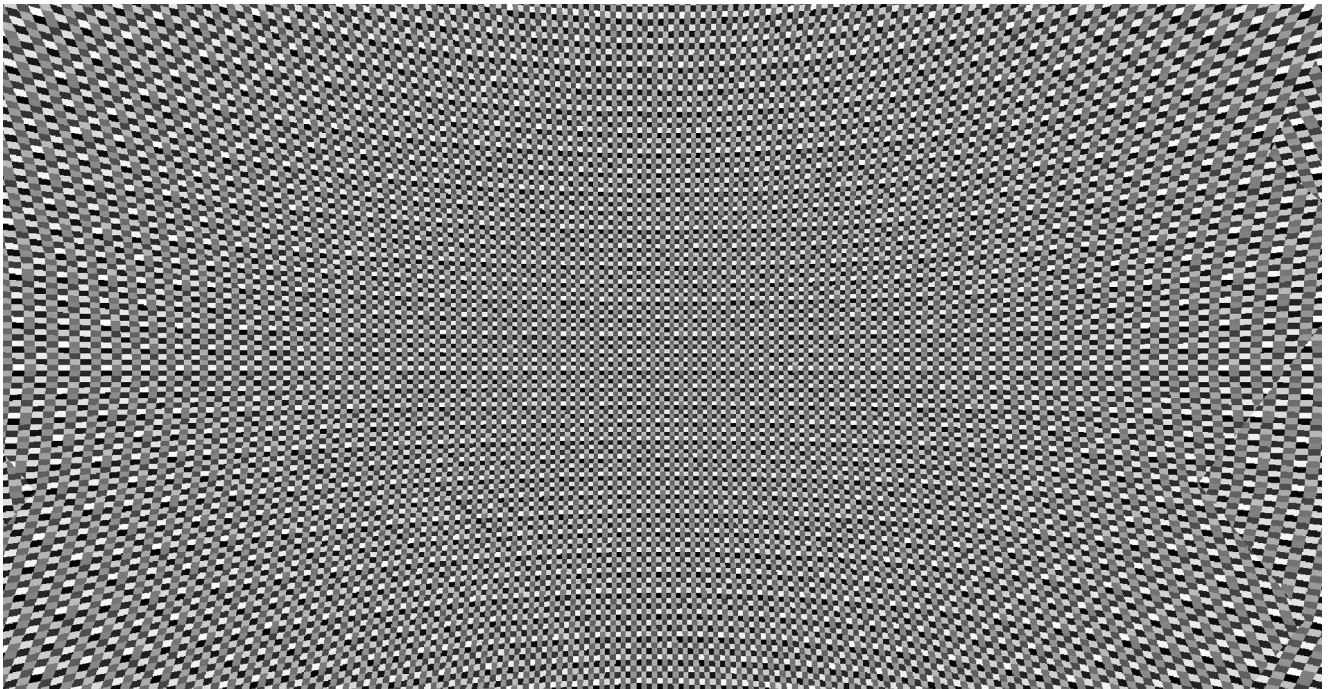
Blue noise dither sphere with a low repetition factor



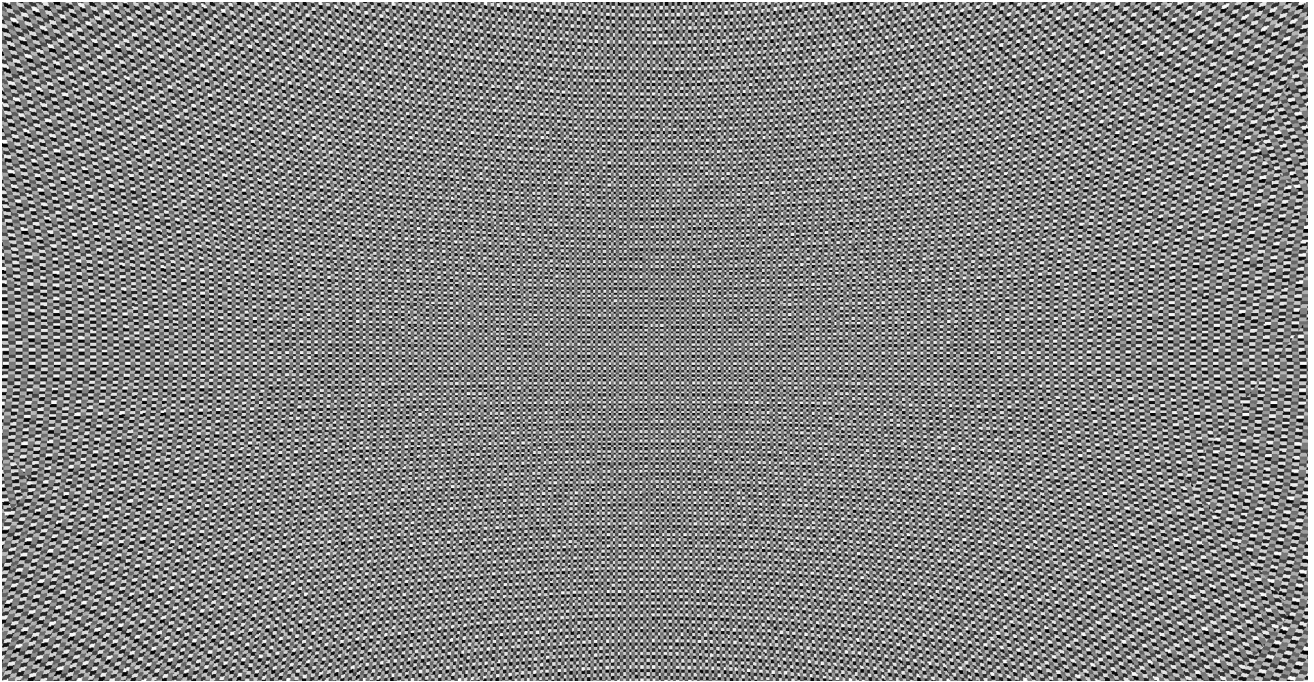
Blue noise dither sphere with a high repetition factor



Bayer dithering with low repetition factor

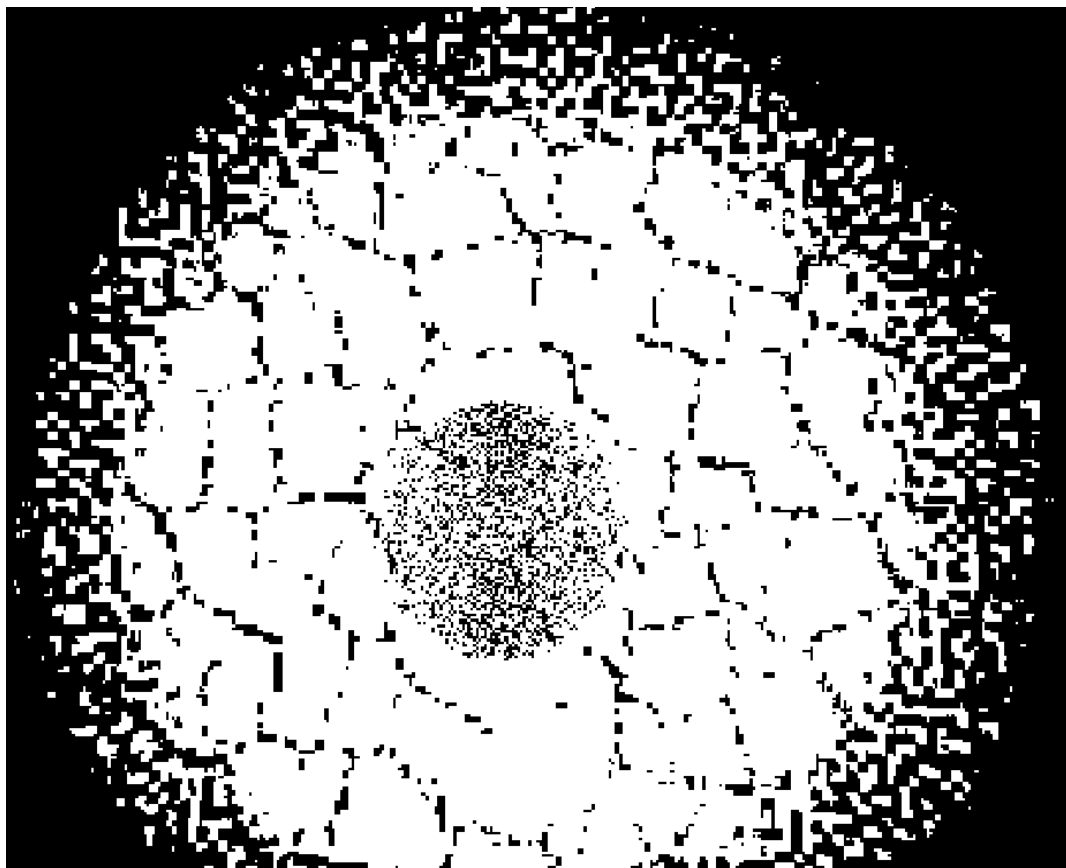


Bayer dithering with high repetition factor

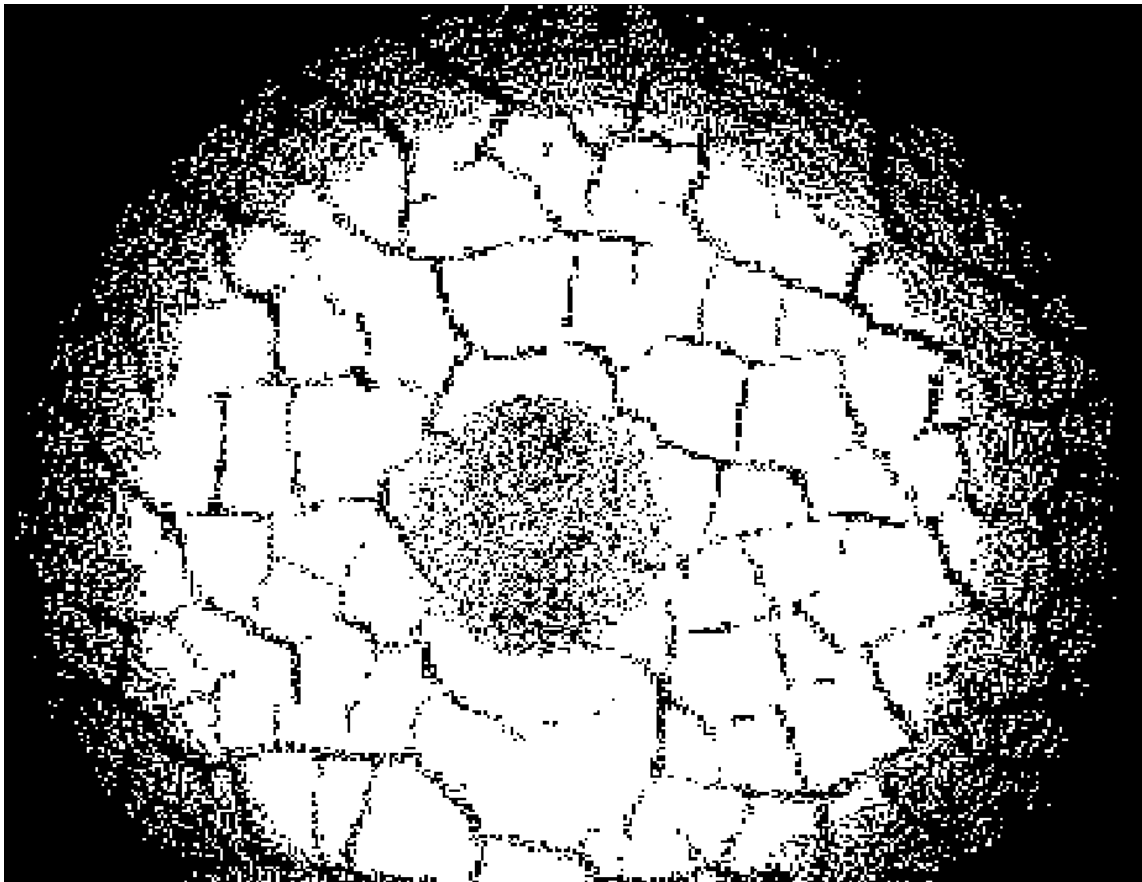


High repetition factors allow yourself to align the dither sphere to the pixels more correctly, but greatly increases the amount of artifacts. Low repetition factors are more stable during camera movements but appear more “out of context” due to the texture’s pixels not corresponding to the screen’s.

Shaded area with low repetition factor



Shaded area with high repetition factor



To avoid aliasing with this method, the dither sphere gets rendered at 2x the render resolution, and then gets downsampled to 1x using supersampling and thresholding to avoid flickering during camera movement.

```
float dither(float diffuse, sampler2D map, float width, float height){
    float min =0.3;
    float max =0.8;
    if (diffuse <=min){
        return 0.0;}
    if (diffuse >=max){
        return 1.0;}
    // 2x2 super sampling for a more stable dithering result, by averaging the dither value of the 4 subpixels
    vec2 xy = vec2(gl_FragCoord.x/width, gl_FragCoord.y/height);
    if (sphere > 0.5){
        vec2 size = vec2(textureSize(map, 0));
        vec2 texel = 1.0 / size;
        // store subpixel dither values in a 2x2 grid around the current pixel
        vec4 values = vec4(0.0);
        values.x = texture(map, xy).r;
        values.y = texture(map, xy + vec2(texel.x, 0.0)).r;
        values.z = texture(map, xy + vec2(0.0, texel.y)).r;
        values.w = texture(map, xy + texel).r;
        // final intensity is based on how many of the subpixel dither values are below the diffuse value, this creates a smoother dithering
        // result and reduces flickering
        float finalIntensity = 0.0;
        finalIntensity += (diffuse > values.x) ? 0.25 : 0.0;
        finalIntensity += (diffuse > values.y) ? 0.25 : 0.0;
        finalIntensity += (diffuse > values.z) ? 0.25 : 0.0;
        finalIntensity += (diffuse > values.w) ? 0.25 : 0.0;
        finalIntensity = finalIntensity > 0.5 ? 1.0 : 0.0;
        // float ditherValue = sum * 0.25;
        return finalIntensity;
    }
    else {
        float threshold = (texture(map, xy)).r;
        float finalIntensity = (diffuse > threshold) ? 1.0 : 0.0;
        return finalIntensity;
    }
}
```


Lighting

To replicate RotOD's basic lighting I implemented point lights and directional lights. Every "Scene" object contains a `std::vector` of "PointLight" objects and a single "DirectionalLight" object. These get properly treated in "LightingPass", accessing them through the scene.

```
// set directional light uniforms
DirectionalLight* dl = &scene->directional_light;
shader->set_uniform3fv("directionalLightDirection", dl->direction);
// shader->set_uniform1f("directionalLightIntensity", dl->intensity);
glActiveTexture(GL_TEXTURE0);
glBindTexture(GL_TEXTURE_2D, *shadow_map.get_depth_buffer());
shader->set_uniform1i("shadowMap", 0);
shader->set_uniformMatrix4fv("lightSpaceMatrix", dl->light_space_matrix);

// set point lights uniforms
shader->set_uniform1i("numPointLights", scene->nl);
shader->set_uniform3fv("cameraWorldPos", scene->camera.Position);
for (int i = 0; i < scene->nl; i++){
    string si = to_string(i);
    string pli = ("pointLightIntensity[" + si + "']").c_str();
    string plp = ("PointLightPos["+si+"']").c_str();
    scene->point_lights[i].set_shader_uniforms(shader, pli, plp);
}
```

"LightingPass" thus includes the shadowmap buffers for the directional light's shadows and renders it before its own render pass:

The start of LightinPass's render function:

```
void render(Scene* scene) override{
    // update shadowmap
    shadow_map.render(scene);
}
```

Shadowmaps are therefore a separate type of RenderPass:

```
class ShadowMap:public RenderPass{
public:
    ShadowMap(int resolution, Shader* shader) {
        create_shadow_map(&this->buffer,&this->depth_buffer, resolution);
        this->render_width = resolution;
        this->render_height = resolution;
        this->shader = shader;
        // this->associated_directional_light = associated_directional_light;
    }
    void render(Scene* scene) override{
        glDepthFunc(GL_LESS);
        glDepthMask(GL_TRUE);
        glBindTexture(GL_TEXTURE_2D, 0);
        glBindFramebuffer(GL_FRAMEBUFFER, buffer);
        glViewport(0, 0, render_width, render_height);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        // render dependant stuff
        shader->Use();
        associated_directional_light = &scene->directional_light;
        glm::mat4 light_space_matrix = associated_directional_light->get_light_space_matrix();
        shader->set_uniformMatrix4fv("lightSpaceMatrix", light_space_matrix);

        // render objects
        for (GLuint i = 0; i < scene->objects.size(); i++) {
            Object* object = &scene->objects[i];
            if (object->material == DUST) continue;
            shader->set_uniformMatrix4fv("modelMatrix", object->model_matrix);
            object->draw();
            glBindTexture(GL_TEXTURE_2D, 0);
        }
    }
}
```

Shader-wise both all the point lights and the directional light use a simple monochrome lambert diffuse, considering there is no need for photorealistic lighting.

Lighting Shader's main function:

```
}
//
void main(){
    float red = 1.0; //lighting
    float green = 1.0; //texture
    float blue = 0.0; // dust?
    float lighting_value = 0.0;
    float tex_value = 1.0;

    vec3 n = normalize(N);
    vec3 l = normalize(L);

    if(textured>=0.9){
        vec2 repeated_UV = mod(interp_UV*repeat, 1.0);
        vec4 surfaceColor = texture(tex, repeated_UV);
        tex_value = length(surfaceColor.rgb);
    }

    //directional light shadowmap
    float shadowFactor=calculateShadow(FragPosLightSpace);
    shadowFactor = smoothstep(0.3,0.6,shadowFactor);
    //directional light diffuse
    float diffDir = max(dot(n,l),0.0);
    float dirLightContrib = diffDir * shadowFactor;

    // point lights
    float pointLightContrib = 0.0;
    for (int i = 0; i < numPointLights; i++) {
        vec4 lightPos = viewMatrix * vec4(PointLightPos[i],1.0);
        vec3 lightDir = (lightPos.xyz - mv.xyz);
        float distance = 2*length(lightDir);
        lightDir = lightDir/distance;
        float diffPoint = max(dot(lightDir,n),0.0);
        // attenuation
        float attenuation = pointLightIntensity[i]/(distance*distance + 1.0);
        float addAmount = (diffPoint*attenuation);
        // addAmount = smoothstep(0.0,0.6,addAmount);
        pointLightContrib += diffPoint*attenuation;
    }

    // combination
    float ambient = 0.2;
    float rawDiffuse = (ambient + dirLightContrib + pointLightContrib);

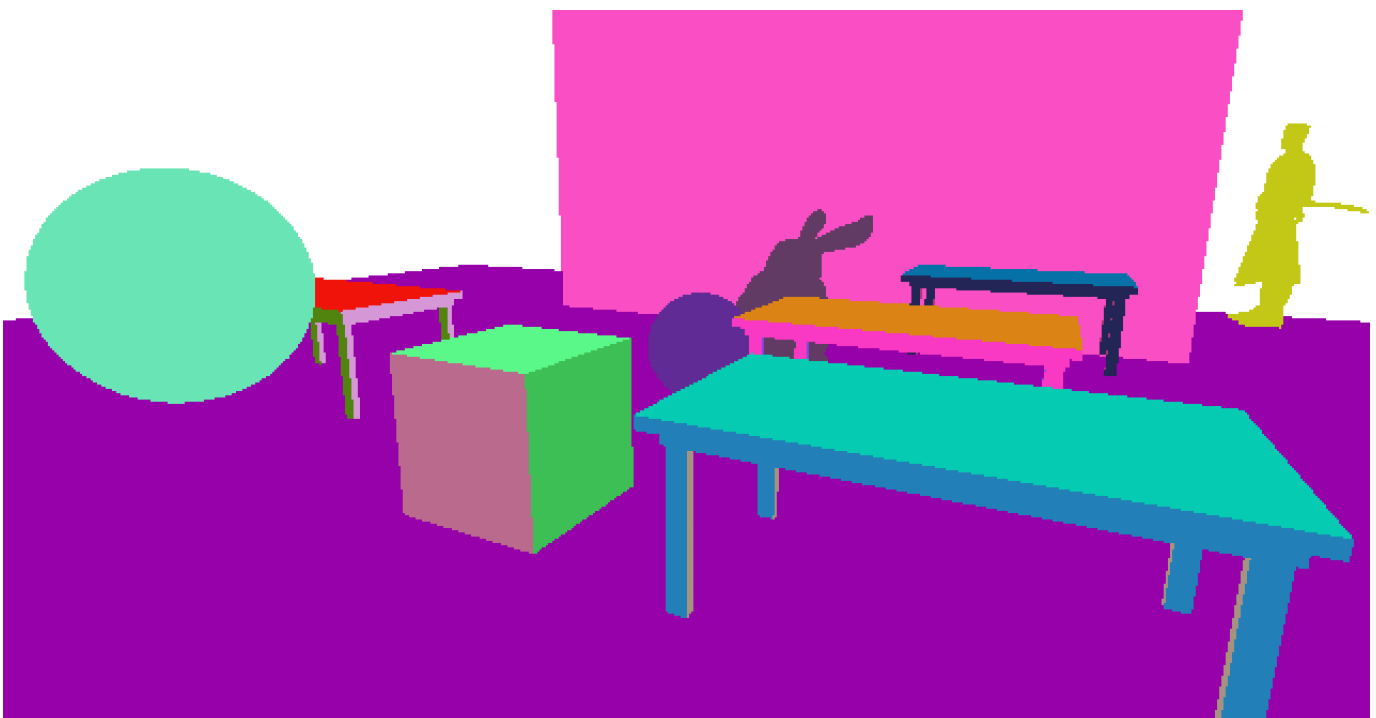
    lighting_value = smoothstep(0.3,0.8,rawDiffuse);
    // lighting_value = floor(lighting_value * 3.0)/3.0;
    lighting_value = lighting_value * tex_value;
    lighting_value = dither(lighting_value, dither_map, width, height);

    // tex_value = smoothstep(0.3,0.8,tex_value);
    // tex_value = floor(tex_value * 4.0)/4.0;
    tex_value = dither(tex_value, dither_map, width, height);

    red = lighting_value;
    green = tex_value;
    blue = 0.0;
    colorFrag = vec4(red,green,blue,1.0);
}
```

Edge accentuation and Outline creation

I implemented RotOD's method for edge accentuation with a few changes. Every face of each model gets assigned a pseudo-random color based on the object's unique ID, the face's normal, and its position in the world. This approximately guarantees a strong contrast between each face that will facilitate further edge detection.



Complex models (with a lot of vertices) are instead rendered with a pseudo-random solid color. In RotOD this is done to have more detailed clothing and facial detail in human models, which is crucial for its gameplay.

```
void main() {
    float floatynumber = 3.3; // just a random float to use in the randomization
    float floatynumber2 = 2.7; // just a random float to use in the randomization
    float f = 1.0 - fill;
    vec3 p = 1+ (pos+100.0)/100.0;
    vec3 n = abs(normalize(N)) + 1;
    n = n*f; // neutralize effect of normal on complex objects
    // randomize color channels based on normal, position, and object_id
    float r = ((fract(n.x * 2.179 + p.x * 2.1123 + n.z * 2.137)/ floatynumber2 ) * floatynumber);
    float g = ((fract(n.z * 2.134 + p.z * 2.1478 + object_id* floatynumber2 ) / floatynumber2 ) * floatynumber);
    float b = ((fract(p.y * 2.122374 + n.y * 2.1398 + p.z * 2.199+ object_id* floatynumber2)/floatynumber) * floatynumber);

    vec3 color = fract(vec3(r, g, b)*33.3333333333);

    colorFrag = vec4(vec3(color), 1.0);
}
```

For outline creation I use a simple laplacian edge detection kernel, which creates 2 pixel wide outlines for every previously accentuated face .

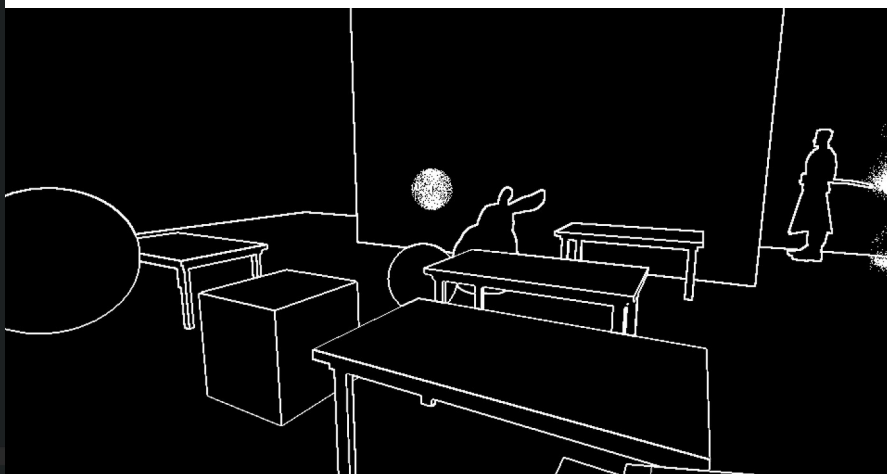
```
// edge detection kernel (Laplacian)
const float kernel[9] = float[9](
    -1, -1, -1,
    -1, 8, -1,
    -1, -1, -1
);

void main() {
    // check if fragment is dust and return white if it is,
    // so that it is not affected by edge detection and remains visible
    vec3 color = texture(edge2_texture, TexCoord).rgb;
    if (color.b >= 1.0 && color.r <= 0.0 && color.g <= 0.0){
        FragColor = vec4(vec3(1.0), 1.0);
        return;
    }

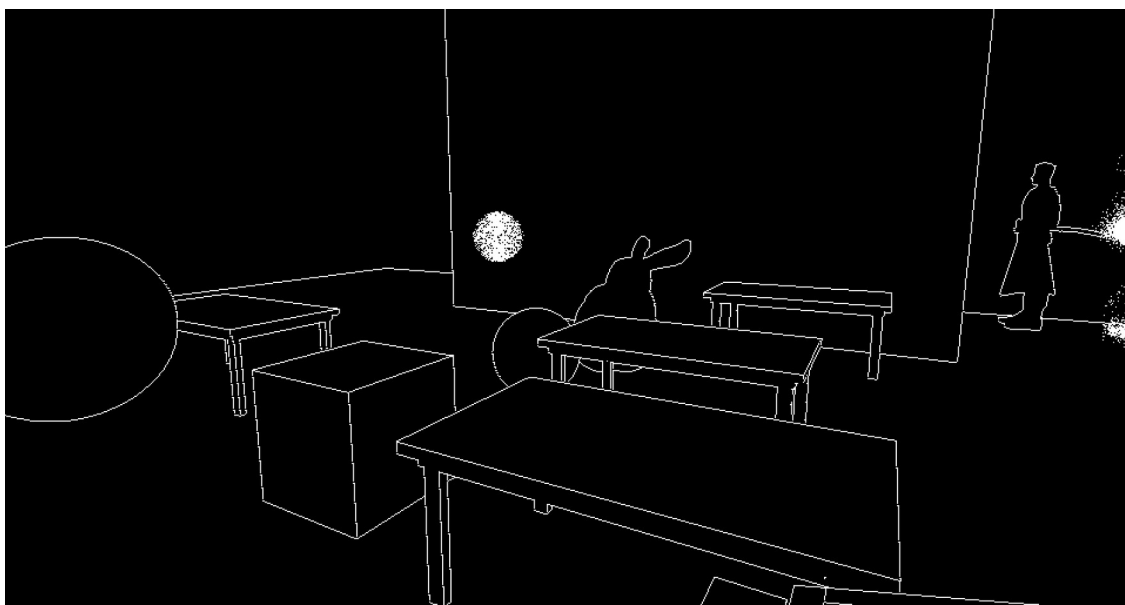
    // apply edge detection to edge1_texture
    vec2 tex_offset = 1.0 / textureSize(edge1_texture, 0);
    vec3 result = vec3(0.0);
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            vec2 offset = vec2(float(i), float(j)) * tex_offset;
            result += texture(edge1_texture, TexCoord + offset).rgb * kernel[(i+1)*3 + (j+1)];
        }
    }

    float edge_strength1 = length(result);

    // soft threshold with antialiasing
    float edge_threshold1 = 0.6;
    float alpha1 = smoothstep(edge_threshold1 - 0.1, edge_threshold1 + 0.1, edge_strength1);
    vec4 result1 = vec4(vec3(alpha1), 1.0); // white on black, intensity varies smoothly
    FragColor = result1;
}
```



As a partially unsuccessful attempt to better recreate RotOD's thinner outlines, I implemented a second outlining method that creates 1 pixel wide outlines with the aid of a depth buffer.



Dust

As an additional material type any scene object can be rendered as a “dust cloud”. Objects with a “DUST” material get rendered using a different draw function that calls “glDrawElementsInstanced” with GL_POINTS, therefore using GPU instancing to render multiple single points in the screen for each vertex of the object’s mesh.

```
void DrawPoints(int n)
{
    // VAO is made "active"
    glBindVertexArray(this->VAO);
    // rendering of data in the VAO
    glDrawElementsInstanced(GL_POINTS, this->indices.size(), GL_UNSIGNED_INT, 0, n);
    // glDrawElements(GL_TRIANGLES, this->indices.size(), GL_UNSIGNED_INT, 0);
    // VAO is "detached"
    glBindVertexArray(0);
}
```

Dust handling in “Object” class:

```
void draw(){
    if (material == DUST)
        this->model.DrawPoints((int)(this->density));
    else
        this->model.Draw();
}
```

“density” is a field of Object used only for dust objects, it controls the number of points drawn for each vertex.

In the edge accentuation’s vertex shader each different point gets pseudo-randomly offset towards the opposite direction of its normal, thus creating a cloud of points that approximate the shape of the mesh that’s being rendered.

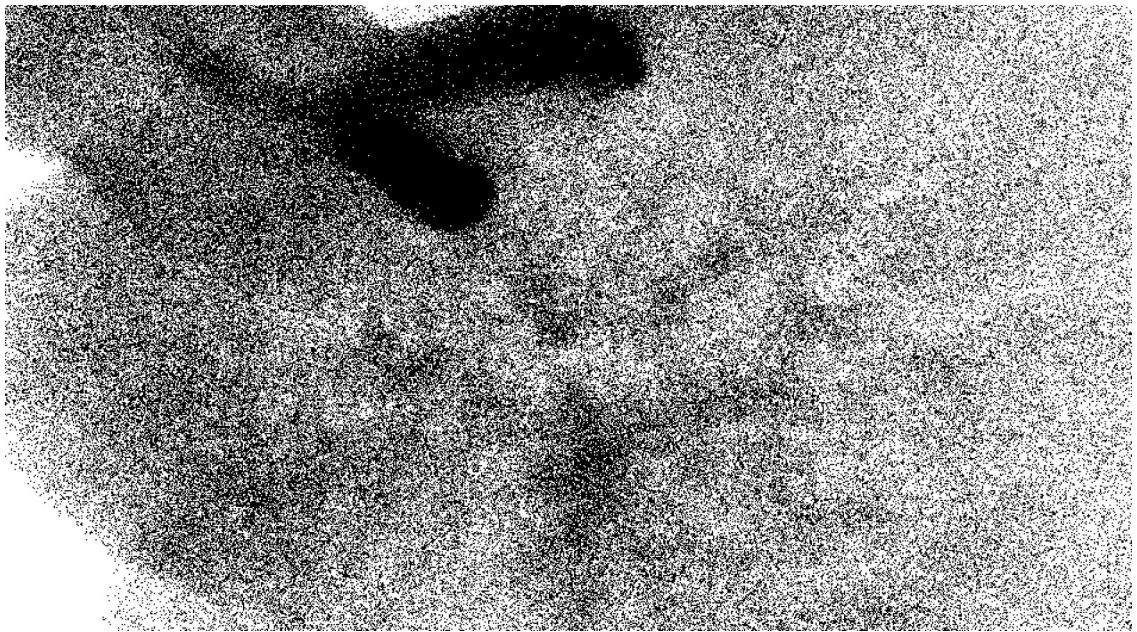
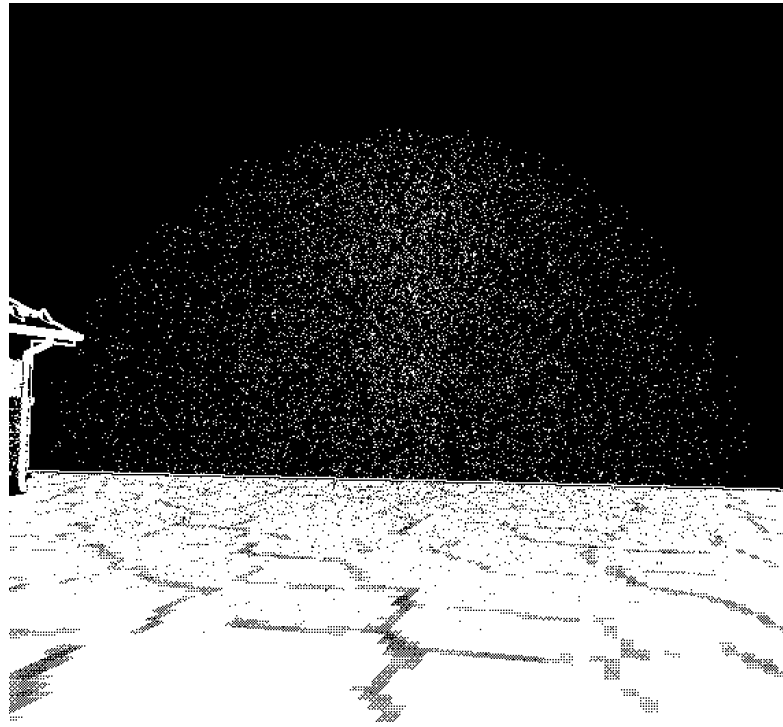
```
} else {
    // Object space data
    vec3 objPos = position; // vertex position (sphere radius = 1 usually)
    vec3 objNorm = normalize(normal); // already normalized
    vec3 objTang = normalize(tangent);
    vec3 objBitang = normalize(cross(objNorm, objTang)); // force orthonormal

    // generate random offsets based on vertex and instance ID
    uint id = uint(gl_VertexID) * 1597 + uint(gl_InstanceID);
    float randRadial = random(id);
    float randTang = random(id + 12345);
    float randBitang = random(id + 67890);
    // range: radial offset moves inward (0 = at surface, 1 = at center)
    float radialAmount = 1.0-(pow(randRadial, 1.0/3.0)); // 0..1, adjust max if needed
    // perpendicular deviation range (in object units, sphere radius = 1)
    float devAmount = 0.3; // tweak this for cloud thickness

    // convert to symmetric range [-devAmount/2, devAmount/2]
    float tOffset = (randTang - 0.5) * devAmount;
    float bOffset = (randBitang - 0.5) * devAmount;

    // new object space position
    vec3 newObjPos = objPos
        - objNorm * radialAmount // move inward
        + objTang * tOffset // random tangent
        + objBitang * bOffset; // random bitangent

    // transform to clip space as usual
    vec4 worldPos = modelMatrix * vec4(newObjPos, 1.0);
    gl_Position = projectionMatrix * viewMatrix * worldPos;
}
```

Performance Evaluation

I measured the frame time and FPS of the project under various specific conditions to test the performance impact of specific features. All the measurements have been taken on my PC with a GTX 1070 Ti and a Ryzen 5 3600 on the linux 24.04 Pop!OS release. The scene I'll use to test the performance contains these objects/models/entities:

- “SIMPLE” objects (faces outlined) of differing polygon count.
 - 3 Plane models - 128 polys
 - 1 Cube models – 12 polys
 - 4 Table model – 42 polys
 - (only high poly exception for edge detection testing) Cottage model - 2171
- 5 “COMPLEX” objects of high polygon count.
 - Cossack soldier – 3829 polys
 - Bunny – 10192 polys
 - 2 Spheres – 400 polys
 - Icosphere for dither sphere rendering - 1280
- 2 (3)“DUST” (dust clouds) objects of different “point” count.
 - 2 Spheres - ~2400 dust points`
 - (used only for benchmarking) Stress test bunny - ~30 million dust points
- 2 Point Lights.
- 1 Directional Light with shadows.

Results

In my test scene where I showcased all these effects and took the screenshot from, I measured

Default parameters = Thick outlines, Dither Sphere enabled

Type of test	Frame time (estimated average over 10 seconds)
Whole scene (Default params)	1.26 ms
Whole scene (Thin outlines, Dither Sphere disabled)	1.37 ms
Whole scene (Thick outlines, Dither Sphere enabled)	1.31 ms
Whole scene (Thin OL, DS enabled, closest to RotOD)	1.45 ms
Whole scene (Default params, Point Lights removed)	1.24 ms
Whole scene (Default params, Shadows removed)	1.16 ms
Whole scene (Default params, Dust Clouds removed)	0.83 ms
Bunny dust cloud stress test (~30 million points)	39.45 ms
Empty Scene (no loaded models)	0.25 ms
Whole scene double (every entity duplicated, Default params)	2.01 ms